Vekto: Verifying Semantic Equivalence of SIMD Kernels Across Architectures

Anonymous author
Anonymous affiliation

1 Introduction

GPUs and tensor accelerators illustrate how hardware specialization can improve the performance and energy efficiency of data-intensive workloads [7]. Exploiting these capabilities has traditionally required substantial human effort: experts must organize data movement, select instructions, and tune kernels for the target hardware. Architecture-specific programming interfaces make this work difficult to reuse when moving to another platform. Recent work has begun to automate parts of this process with language-model agents that generate and optimize kernels using hardware knowledge and execution feedback [12, 7].

Specialized execution units are also embedded in general-purpose CPUs, where SIMD and vector extensions accelerate data-parallel computation. Their execution models differ substantially: Arm NEON operates on fixed-width vectors, whereas RISC-V Vector (RVV) allows the active vector length to be selected at runtime [2, 15]. A migration between them must account for different loop structures, tail handling, and instruction behavior. RVV is a more recent addition to this ecosystem, and extending its kernel support has motivated work on porting existing vector code [5, 6]. C intrinsic interfaces expose these hardware operations within ordinary C programs. Programmers can combine C control flow and pointer operations with vector intrinsics that the compiler lowers to target instructions, avoiding the need to implement an entire kernel in assembly.

This programming interface also gives agents a way to apply their code-generation capabilities to SIMD optimization. Intrinsic descriptions and hardware-specific guidance can inform changes to instruction selection, vector grouping, and loop organization [5, 7]. However, these changes can alter program behavior on inputs that the available tests do not exercise. Reports of reward hacking in kernel-generation benchmarks further demonstrate that an optimization objective can be satisfied by exploiting weaknesses in its evaluation [8]. Existing work addresses correctness through testing and formal validation, including bounded equivalence checking of translated kernels and verification of vectorization in compiler IR [5, 17]. Establishing equivalence for the complete calling domain requires reasoning about the kernels' loops and memory accesses while accounting for numerical differences between their ISAs. We seek this guarantee for the actual optimized implementations, including the control flow and intrinsic choices responsible for their performance.

A shared semantic basis for this verification can be constructed at several levels. At the instruction level, ISA specifications describe operations on architectural state, enabling machine-code proofs grounded in detailed hardware semantics [3, 16]. At the intermediate-representation level, vector operations can be expressed in an IR and compared using its computational and memory semantics. LLVM translation validation and verified instruction selection for Hexagon HVX demonstrate uses of this approach [10, 1]. Intrinsic source code provides a third option: interpreting supported intrinsics as C operations turns the comparison into a C program verification problem. The choice determines how vector behavior is connected to the surrounding control flow and memory accesses, and which semantic definitions can be reused across kernels.

Our verification target is functional equivalence under explicit calling contracts: the two kernels must agree on their returned outputs and satisfy their memory specifications for admitted inputs and hardware configurations. For this target, the intrinsic interface

exposes the vector operations together with the C loops and pointers that manipulate them. Reasoning from compiled instructions would also require relating architectural states and connecting the machine-code behavior to the source kernels. We instead interpret the intrinsics at the C level, where their numerical and memory effects can be inspected alongside the source program and handled by existing C verification infrastructure. This choice preserves architecture-specific behavior in the intrinsic definitions while giving the complete kernels a common language for verification. The correspondence between those definitions and native instructions remains an explicit assumption of the approach.

We present Vekto, a framework for cross-ISA kernel equivalence checking through reusable C semantic shims, with NEON, RVV, AVX, and Hexagon HVX as its target interfaces. [**TODO:** Report the supported intrinsic subsets.] For each kernel pair, Vekto binds intrinsic calls to their semantic definitions and states the required output relation in a common specification. QCP symbolically executes the annotated C programs and generates verification conditions for memory accesses, loop invariants, and postconditions, which are discharged through automation and Rocq/Coq proofs [18]. Each implementation is proved against the common specification using its own invariants, so their loops need not advance in lockstep. Implementations, specifications, and proofs remain separate, allowing agents to develop annotations and auxiliary lemmas while retaining the executable kernels. The resulting guarantee is partial correctness relative to the calling contracts and semantic models (Section 3).

Human review centers on the calling contract and the evidence produced during verification. A concrete counterexample must first be checked against the original caller's assumptions. If those assumptions exclude the input, the specification can be strengthened to express them, with evidence that all legitimate calls remain covered. If the input is feasible, it identifies a behavioral discrepancy for target-side repair. The revised implementation is then verified again and benchmarked to assess the repair's performance cost. An unsuccessful proof attempt can also reflect a missing invariant or lemma, so unresolved obligations remain distinct from demonstrated discrepancies. Keeping proof changes separate from executable changes makes it possible to review the reason for each repair and its effect on the optimized code.

This equivalence framework supports three uses. First, it checks whether an optimized kernel preserves its reference behavior, within one ISA or across ISAs. Second, it supplies a correctness criterion for porting existing NEON or AVX kernels to RVV or HVX libraries that lack the corresponding operation. Existing implementations then provide both source code for translation and a reference for verification. Third, it enables semantic cross-checking of independently developed implementations of the same operation. As in differential testing, disagreement provides a starting point for investigation; equivalence checking makes the relationship between the implementations an explicit proof obligation [11, 9]. Examining a concrete discrepancy against the shared contract can identify an implementation defect or an allowed numerical difference, guiding repairs and clarifications to the library's documented behavior.

Our planned evaluation covers these uses through 35 final optimized RVV kernels from SALTyRN, five AVX operations to be ported to RVV and HVX, and corresponding implementations in XNNPACK [4]. A composed SIMD program, with softmax as a candidate, will examine whether the kernel specifications support equivalence reasoning across operation boundaries. We will measure verification coverage, proof-development time and token cost, the changes required for verification or repair, and performance against consistent target-side baselines. [**TODO:** Report verified pairs and speedups, completed ports, cross-checking

NEON: fixed-width main loop

```
for (; batch >= 4 * sizeof(float); batch -= 4 * sizeof(float)) {
    const float32x4_t va = vld1q_f32(input_a); input_a += 4;
    const float32x4_t vb = vld1q_f32(input_b); input_b += 4;

    float32x4_t vacc = vmaxq_f32(va, vb);

    vst1q_f32(output, vacc); output += 4;
  }
```

RVV: full blocks and a variable-length tail

```
  size_t n = batch / sizeof(float);
  const size_t vlmax = __riscv_vsetvlmax_e32m8();
  while (n >= vlmax) {
    vfloat32m8_t va = __riscv_vle32_v_f32m8(input_a, vlmax);
    vfloat32m8_t vb = __riscv_vle32_v_f32m8(input_b, vlmax);
    vfloat32m8_t vacc = __riscv_vfmax_vv_f32m8(va, vb, vlmax);
    __riscv_vse32_v_f32m8(output, vacc, vlmax);
    input_a += vlmax;
    input_b += vlmax;
    output += vlmax;
    n -= vlmax;
  }
  if (n > 0) {
    size_t vl = __riscv_vsetvl_e32m8(n);
    vfloat32m8_t va = __riscv_vle32_v_f32m8(input_a, vl);
    vfloat32m8_t vb = __riscv_vle32_v_f32m8(input_b, vl);
    vfloat32m8_t vacc = __riscv_vfmax_vv_f32m8(va, vb, vl);
    __riscv_vse32_v_f32m8(output, vacc, vl);
  }
```

Figure 1 Excerpts from the unchanged source implementations. Signatures, assertions, and the NEON tail are omitted here; the complete source files are retained in the draft package. The RVV comments are omitted without changing executable statements. `batch` counts bytes, and `n` counts float elements. The proof must cover the complete functions, including the omitted tail.

findings and their upstream status, and the composed-program result.] These experiments will assess the extent to which the framework can validate optimized code, reuse semantic definitions across implementations, and support repairs that retain the benefit of optimization.

2 A motivating example: maximum across two vector ISAs

[**TODO:** Candidate example. The discrepancy and diagnostic measurements below have archived evidence. A complete proof of the repaired final optimized kernel is pending. This example does not establish a speedup over XNNPACK and therefore does not yet satisfy the intended performance narrative.]

Consider an elementwise maximum kernel that reads two binary32 arrays and writes one result per element. Figure 1 shows excerpts from a NEON reference and the final RVV implementation in SALTyRN's published optimized-kernel repository [4]. The NEON loop processes four elements at a time. Its tail loads another four elements but stores only the requested prefix. The RVV implementation uses the maximum active length for full blocks and selects a shorter length for the final remainder. These control flows require different arguments about the processed prefix and valid memory accesses. In particular, the NEON tail requires readable padding even when only one output element is requested.

The numerical operations also differ. For the Arm floating-point profile used in this

```c
unsigned int qcp_fmax_arm_bits(unsigned int a,unsigned int b)
{
 if (qcp_fmax_snan_bits(a) == 1) return a | 4194304U;
 if (qcp_fmax_snan_bits(b) == 1) return b | 4194304U;
 if (qcp_fmax_nan_bits(a) == 1) return a;
 if (qcp_fmax_nan_bits(b) == 1) return b;
 return qcp_fmax_ordered_bits(a,b);
}

unsigned int qcp_fmax_rvv_bits(unsigned int a,unsigned int b)
{
 if (qcp_fmax_nan_bits(a) == 1) {
  if (qcp_fmax_nan_bits(b) == 1) return 2143289344U;
  return b;
 }
 if (qcp_fmax_nan_bits(b) == 1) return a;
 return qcp_fmax_ordered_bits(a,b);
}
```

Figure 2 Existing C lane interpretations, with their QCP annotations omitted for space. The helpers inspect binary32 bit patterns and compare non-NaN values. Decimal constants `4194304U` and `2143289344U` encode the quiet-NaN bit and a canonical quiet NaN. The original definitions and contracts accompany the excerpts. The modeled Arm profile disables flush-to-zero and default-NaN behavior; floating-point exception flags are outside the output observation.

example, `vmaxq_f32` propagates a NaN operand. RVV's `vfmax` returns the numerical operand when exactly one operand is NaN [2, 15]. Treating both instructions as a single mathematical maximum would erase the difference before any program proof began. The semantic shims instead retain separate lane functions. Figure 2 extracts these functions from the existing QCP shim: the Arm branch quiets signaling NaNs and preserves a NaN result, whereas the RVV branch selects the numerical operand or a canonical NaN. Vector shims apply the appropriate function to the active lanes. This code illustrates existing semantic definitions, not a newly completed hardware-refinement proof.

The disagreement has a one-element witness:

$$n = 1, \qquad A[0] = \texttt{0x7fc12345}\ \text{(quiet NaN)}, \qquad B[0] = \texttt{0x3f800000}\ (1.0).$$

With sufficient input padding and disjoint buffers, executing the compiled kernels under AArch64 and RVV instruction emulation produces NaN and 1.0, respectively. The RVV result is the same at the three tested hardware vector lengths, 128, 256, and 512 bits. This is a concrete execution discrepancy. It is not a difference in NaN payload or the sign of zero, both of which the output relation in Section 3 permits.

A diagnostic run of the local SALTyRN bounded checker accepted the same final RVV candidate at input length one and VLEN 512. Inspection showed that its NEON maximum model used the same floating-point maximum operation as the RVV model. In an isolated copy, adding the NEON NaN-propagation case changed the result to a counterexample without changing the kernel pair, input length, or comparison relation. Thus this particular missed discrepancy concerns the semantic model, not the input-length bound. The run used a locally adapted checker and is not a reconstruction of the exact verification run reported in SALTyRN's paper. It also does not demonstrate that QCP automatically synthesizes counterexamples.

To preserve the NEON output behavior, the candidate repair retains the RVV maximum and selects a canonical NaN wherever either operand is NaN:

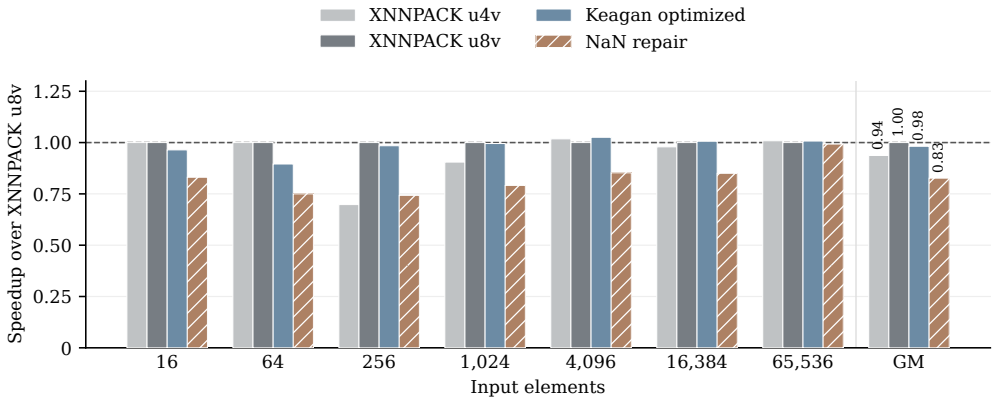


Figure 3 Preliminary local Saturn RTL measurements on finite inputs, reproduced from the archived experiment. Speed is XNNPACK RVV u8v cycles divided by the measured implementation's cycles. Each size uses one warm-up and the median of three timed calls within one simulation run; no confidence interval is claimed. Hardware VLEN is 512 bits and datapath width is 256 bits. The published optimized and XNNPACK implementations do not preserve the NEON NaN behavior; the repair adds that behavior. These are measurements, not proof results or a precise reproduction of the SALTyRN paper's configuration.

```
vbool4_t a_nan = __riscv_vmfne_vv_f32m8_b4(va, va, vl);
vbool4_t b_nan = __riscv_vmfne_vv_f32m8_b4(vb, vb, vl);
vbool4_t any_nan = __riscv_vmor_mm_b4(a_nan, b_nan, vl);
vacc = __riscv_vfmerge_vfm_f32m8(
    vacc, __builtin_nanf(""), any_nan, vl);
```

The repair is inserted after maximum in both the full-block and tail paths, using `vlmax` in the former. It leaves the NEON implementation, RVV loop organization, and register grouping unchanged. Regression executions agree under the intended output relation, including mixed exceptional and finite values. [**TODO:** Complete the final optimized kernel's shim bindings, loop and memory obligations, and all dependent proof checks for the same profile as the performance experiment. Historical proofs of an intermediate RVV implementation do not discharge these obligations.]

The measured geometric-mean speeds relative to XNNPACK u8v are 0.982× for the published optimized candidate and 0.826× for the repair (Figure 3). The repair increases the geometric mean of the per-size cycle ratios by 18.8%. These measurements do not support a claim that this candidate retains a performance advantage over XNNPACK. They instead expose the cost of preserving the selected reference behavior. Whether that behavior is required depends on the contract: XNNPACK's historical min/max discussion explicitly declines a NaN-handling guarantee [19]. Consequently this witness establishes failure of NEON-preserving equivalence, without by itself establishing an XNNPACK API defect.

3 Contracts, observations, and proof obligations

3.1 The relation being checked

Let x be a logical input, including array contents, dimensions, and operation parameters. A domain predicate $D(x)$ describes legitimate calls. For implementation K_i, a precondition $P_i(x, s_i)$ relates x to the initial C state s_i, including its memory representation. A machine

profile p_i records the ABI, relevant floating-point configuration, and supported vector configuration. The two profiles need not use the same vector width or state representation.

For integer results we require exact equality. For the floating-point outputs considered here, define

$$u \approx_{\mathrm{fp}} v \quad \Longleftrightarrow \quad u =_{\mathrm{num}} v \ \vee \ (\mathsf{isNaN}(u) \wedge \mathsf{isNaN}(v)). \tag{1}$$

Numeric equality identifies positive and negative zero. This relation also identifies NaN payloads, but permits no tolerance between distinct finite numbers. We lift it pointwise to output arrays, requiring their shapes to agree. A different observation must be stated explicitly for a kernel whose caller inspects bits, indices, or additional state.

Write $\langle K_i, s_i \rangle \Downarrow_{p_i} t_i$ for a terminating modeled execution. The partial-correctness comparison is

$$\begin{aligned} &D(x) \wedge P_A(x, s_A) \wedge P_B(x, s_B) \ \wedge \\ &\langle K_A, s_A \rangle \Downarrow_{p_A} t_A \ \wedge \langle K_B, s_B \rangle \Downarrow_{p_B} t_B \quad \Longrightarrow \quad \mathsf{obs}(t_A) \approx \mathsf{obs}(t_B). \end{aligned} \tag{2}$$

Here obs extracts the specified output and return values. Memory safety and preservation of memory outside the permitted write footprint are separate obligations of the implementation contracts. They do not follow from Equation 2 alone. The current QCP proof packages inspected for this draft explicitly exclude a termination theorem. Accordingly, Equation 2 does not assert that either implementation returns, or that they agree on divergence. **[TODO:** If the final contribution includes total correctness, add and check termination arguments for the admitted implementations; do not infer them from return VCs.]

3.2 Relating implementations through a common specification

Each implementation is checked against a postcondition that relates its output to a common relation $S(x, o)$. To compose the results into equivalence, the common relation must determine observations up to the selected equivalence:

$$S(x, o_1) \wedge S(x, o_2) \Longrightarrow o_1 \approx o_2. \tag{3}$$

This condition prevents a weak common specification, such as a broad error interval, from being mistaken for an equivalence proof. For a reference-preserving floating-point optimization, S may encode the reference's rounding and accumulation order rather than ideal real arithmetic. The proof must establish the result for each complete implementation, including helpers and tails. A wrapper that assumes both kernel contracts only establishes the final composition conditional on those contracts.

The relation in Equation 1 is a final-output observation. It cannot be substituted for bit-level equality at every intermediate boundary: a subsequent reciprocal or sign-sensitive operation can distinguish positive from negative zero. A composed program therefore needs intermediate contracts strong enough for each consumer, or a proof that the consumer respects the chosen relation.

3.3 What QCP checks

QCP processes contracts, assertions, and loop invariants by symbolic execution in separation logic [18, 13]. In the inspected VADD package, the public contract supplies readable input arrays, additional padding for NEON tail loads, and a separate writable output array. It returns the original input resources and a fully initialized output array with specified values.

Table 1 Planned studies. Subject counts specify the intended evaluation, not proof successes. Softmax is a candidate composed program, not a full inference application.

Study	Subjects	Planned scope	Completed proofs
Optimization	Final SALTyRN targets	35 pairs	[**TODO:** pending]
Translation	AVX operations to RVV/HVX	5 operations	[**TODO:** pending]
Cross-checking	Corresponding XNNPACK kernels	[**TODO:** select]	[**TODO:** pending]
Composition	A small SIMD program	[**TODO:** select]	[**TODO:** pending]

At an indexed read, generated entailments expose the cell at the requested index from the array predicate and retain the remainder of the array. Other conditions establish expression ranges, satisfy callee preconditions, preserve loop invariants, and derive the postcondition on return. Memory-access validity is checked through these resources and entailments as well as symbolic-execution failures; it is not confined to obligations named `safety` [14].

The proof package checks generated `VC_Correct` interfaces for the kernel bodies, shims, helpers, and final comparator. Statements declared inside such an interface are obligations to implement, not evidence that the obligations have been proved. The implementation/specification binding and dependency checks are therefore part of accepting a result. The inspected package establishes modeled C-shim partial correctness and explicitly excludes hardware-to-shim correspondence and termination. [**TODO:** Freeze the release and complete the per-kernel coverage audit for the final experimental cohort.]

3.4 Review and trust boundaries

Reusing a semantic shim requires a matching operation, type, and machine profile. In particular, a proof at one fixed RVV capacity is not a proof for all hardware VLENs. A sound generalization must account for the allowed active-length choices and the behavior of inactive or masked lanes. Source preparation must also preserve executable statements and their interpretation. Compiler correctness and the correspondence from modeled intrinsic semantics to native execution remain outside the present proof chain. The QCP paper reports that its symbolic executor lacked a formal soundness proof at publication [18]; [**TODO:** audit the exact QCP release used in the final artifact and state its remaining trusted components.]

When strengthening a precondition from P to P', let R describe the original caller's legitimate states. Acceptance requires evidence for $R \Rightarrow P'$, not merely that P' excludes one counterexample. Likewise, changing a target creates a new candidate whose proof and timing evidence must be rebound to the changed source. Source edits, contract changes, and proof-only changes are recorded separately. A failed proof, a replayed counterexample, and a completed proof are distinct outcomes.

4 Evaluation design

[**TODO:** This section specifies planned experiments. It contains no completed cohort-level verification or performance results.]

The primary cohort is fixed to the 35 published final optimized RVV kernels at commit `80feb4564cb4` [4]. The study evaluates existing optimization outputs; it does not rerun the optimization search. For each subject, the matching NEON source, original optimized target, any repair, caller contract, and machine profile must be identified separately. Earlier proofs of intermediate targets are reusable development assets, not completed results for this cohort.

Table 2 Per-kernel reporting schema. Each row must identify exact source versions and the checked profile in the artifact. Times include failed attempts where applicable. No placeholder is a measurement.

Field	Required value
Proof outcome	Proved, concrete discrepancy, or unresolved; [**TODO:** counts]
Contract/profile	Admitted lengths, values, memory layout, VLEN and FP modes
Executable changes	Added/deleted lines by ISA, rationale, review decision
Verification cost	Wall time, generation/checking time, tokens, retries; [**TODO:** measure]
Reuse	Reused intrinsic lemmas and assumptions; [**TODO:** measure incremental cost]
Performance	Baseline/original/repaired cycles and speed ratios; [**TODO:** measure]
External optimization cost	Upstream records, or explicitly not reported

Measurements should distinguish proof construction from proof checking. Per-kernel records include elapsed time, QCP condition generation, proof checking, agent token usage, failed attempts, manual interventions, and implementation changes. Shared semantic-library construction is reported separately from incremental kernel work. Historical optimization time and tokens are reported only when traceable upstream records exist. The study compares original optimized and repaired candidates against the same handwritten RVV baseline, using identical input sizes, compiler settings, and target hardware configuration. An invalid candidate's timing may quantify repair cost, but is not counted as a valid speedup.

The extension study must establish that selected operations are absent from the chosen target-library versions rather than infer absence from filenames. Cross-checking must align logical operations, layouts, parameters, and numerical contracts before classifying a discrepancy. Existing XNNPACK reports supply potential historical regression subjects, including RVV rounding and square-root exceptional-value handling [20, 22]. They are known reports, not new findings by Vekto. For softmax, the comparison must address reduction order, exponential approximations, intermediate-buffer contracts, and the final division. [**TODO:** Select a real source program and target baseline, and establish whether the desired exact observation is compatible with their algorithms.]

Three case studies are planned: a minimally repaired optimized kernel; a new port that requires additional intrinsic semantics; and a cross-checking discrepancy whose feasibility and contract implications can be traced to concrete code. [**TODO:** Choose completed cases with adequate provenance. The maximum example currently supports semantic diagnosis, but not the desired retained-speedup claim.]

5 Related work

[**TODO:** Focused initial comparison. Complete the literature coverage before making a novelty claim.]

Generation and validation of optimized kernels.

KernelBench, Autocomp, and IntrinTrans use executable feedback to evaluate or improve generated kernels [12, 7, 6]. SALTyRN also includes bounded symbolic comparison before and after optimization [5]. LLM-Vectorizer combines generation with Alive2 validation and describes loop-alignment techniques and restrictions for its experiments [17]. These systems

motivate comparing the input domain, semantic model, and complete program obligations of each guarantee, rather than grouping all prior validation as testing. VEKTO's proposed evaluation asks whether modular C proofs can cover the actual optimized kernels under explicit contracts.

Semantic models and verified transformations.

Sail and Islaris provide a path from authoritative ISA specifications to machine-code reasoning [3, 16]. Alive2 checks LLVM refinement with bounded reasoning about loops [10]. Rake checks equivalence of synthesized vector instruction sequences for Hexagon HVX [1]. These operate at different boundaries from VEKTO's C intrinsic interface. The proposed method uses QCP's existing C verification infrastructure [18]; reusable cross-ISA semantic definitions, the alignment of kernel contracts, and evidence about optimized implementations must supply the contribution beyond that infrastructure.

Cross-checking implementations.

Differential testing uses disagreements among implementations to identify candidate defects [11]. SymDiff checks equivalence and exposes behavioral differences between imperative programs [9]. Our proposed XNNPACK study applies an explicit observation relation to corresponding ISA implementations, while retaining the distinction between disagreement and violation of a required contract. The distinction has practical consequences: a historical XNNPACK min/max policy permits unspecified NaN behavior [19], while a later change to the repository's separate YNNPACK subsystem adds NaN propagation for a reported JAX reduction problem [21]. That change is evidence of an integration concern, not proof that our tested XNNPACK microkernels violate their API.

References

1 Maaz Bin Safeer Ahmad, Alexander J. Root, Andrew Adams, Shoaib Kamil, and Alvin Cheung. Vector instruction selection for digital signal processors using program synthesis. In *Architectural Support for Programming Languages and Operating Systems*, 2022. URL: `https://people.eecs.berkeley.edu/~akcheung/papers/asplos22.pdf`, `doi:10.1145/3503222.3507714`.

2 Arm. Arm Neon intrinsics reference. Arm C Language Extensions; accessed November 4, 2026. URL: `https://arm-software.github.io/acle/neon_intrinsics/advsimd.html`.

3 Alasdair Armstrong et al. ISA semantics for ARMv8-A, RISC-V, and CHERI-MIPS. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019. URL: `https://alastairreid.github.io/papers/popl19-isasemantics.pdf`, `doi:10.1145/3290384`.

4 Keagan Chern. Optimized RVV kernels, 2026. Published final-output source repository, commit 80feb4564cb470de301062109b6f89db672bdf9d. URL: `https://github.com/keaganchern/optimized-rvv-kernels/tree/80feb4564cb470de301062109b6f89db672bdf9d`.

5 Keagan Chern, Charles Hong, Sophia Shao, and Alvin Cheung. SALTyRN: Turning Neon kernels into fast, verified RVV code with LLMs. RISE Project, author technical article, July 27, 2026. URL: `https://riseproject.dev/2026/07/27/saltyrn-turning-neon-kernels-into-fast-verified-rvv-code-with-llms/`.

6 Liutong Han, Zhiyuan Tan, Hongbin Zhang, Pengcheng Wang, Chu Kang, Mingjie Xing, and Yanjun Wu. IntrinTrans: LLM-based intrinsic code translator for RISC-V vector. arXiv:2510.10119, version 3, 2026. URL: `https://arxiv.org/abs/2510.10119v3`.

7 Charles Hong, Sahil Bhatia, Alvin Cheung, and Yakun Sophia Shao. Autocomp: LLM-driven code optimization for tensor accelerators. arXiv:2505.18574, version 3, 2025. URL: `https://arxiv.org/abs/2505.18574v3`.

8 Natalia Kokoromyti, Sokserey Sun, Anne Ouyang, and Azalia Mirhoseini. KernelBench v0.1. Scaling Intelligence Lab, Stanford University. Maintainer report on evaluation and reward hacking; accessed November 4, 2026. URL: https://scalingintelligence.stanford.edu/blogs/kernelbenchv01/.

9 Shuvendu K. Lahiri, Chris Hawblitzel, Ming Kawaguchi, and Henrique Rebelo. SymDiff: A language-agnostic semantic diff tool for imperative programs. In *Computer Aided Verification*, 2012. URL: https://www.microsoft.com/en-us/research/publication/symdiff-a-language-agnostic-semantic-diff-tool-for-imperative-programs/.

10 Nuno P. Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: Bounded translation validation for LLVM. In *Programming Language Design and Implementation*, 2021. URL: https://web.ist.utl.pt/nuno.lopes/pubs/alive2-pldi21.pdf.

11 William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998. URL: https://www.cs.tufts.edu/comp/150FP/archive/bill-mckeeman/DifferentailTesting.pdf.

12 Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? In *International Conference on Machine Learning*, 2025. URL: https://proceedings.mlr.press/v267/ouyang25a.html.

13 Qualified C Programming. QCP tutorials. Accessed November 4, 2026. URL: https://www.qua.codes/en/tutorials/index.html.

14 Qualified C Programming. Symbolic execution generates verification conditions. Tutorial T3-3; accessed November 4, 2026. URL: https://www.qua.codes/en/tutorials/t3-3.html.

15 RISC-V International. The RISC-V instruction set manual: V standard extension for vector operations, version 1.0, 2026. ISA manual release v20260120. URL: https://docs.riscv.org/reference/isa/v20260120/unpriv/v-st-ext.html.

16 Michael Sammler, Angus Hammond, Rodolphe Lepigre, Brian Campbell, Jean Pichon-Pharabod, Derek Dreyer, Deepak Garg, and Peter Sewell. Islaris: Verification of machine code against authoritative ISA semantics. In *Programming Language Design and Implementation*, 2022. URL: https://pub.ista.ac.at/~msammler/paper/islaris.pdf, doi:10.1145/3519939.3523434.

17 Jubi Taneja, Avery Laird, Cong Yan, Madan Musuvathi, and Shuvendu K. Lahiri. LLM-Vectorizer: LLM-based verified loop vectorizer. arXiv:2406.04693, version 1, 2024. URL: https://arxiv.org/abs/2406.04693v1.

18 Xiwei Wu, Yueyang Feng, Xiaoyang Lu, Tianchuan Lin, Kan Liu, Zhiyi Wang, Shushu Wu, Lihan Xie, Chengxi Yang, Hongyi Zhong, Naijun Zhan, Zhenjiang Hu, and Qinxiang Cao. NEAT: QCP: A practical separation logic-based C program verification tool. arXiv:2505.12878, version 1, 2025. URL: https://arxiv.org/abs/2505.12878v1.

19 XNNPACK contributors. Handling NaN for min/max instruction, 2020. Issue 441; includes subsequent discussion. URL: https://github.com/google/XNNPACK/issues/441.

20 XNNPACK contributors. vrnd kernels for rvv are incorrect, 2025. Issue 8087; later discussion reports a merged fix. URL: https://github.com/google/XNNPACK/issues/8087.

21 XNNPACK contributors. Correctly propagate NaN in min/max operations, 2026. Pull request 11652, merged October 14, 2026; changes the YNNPACK subsystem. URL: https://github.com/google/XNNPACK/pull/11652.

22 XNNPACK contributors. Fix f32-vsqrt and f16-vsqrt inf handling, 2026. Pull request 11053, merged August 25, 2026. URL: https://github.com/google/XNNPACK/pull/11053.